ARITIFICIAL INTELLIGENCE(Q) – MAY 2026
KECERDASAN TIRUAN(Q) – *MEI 2026*

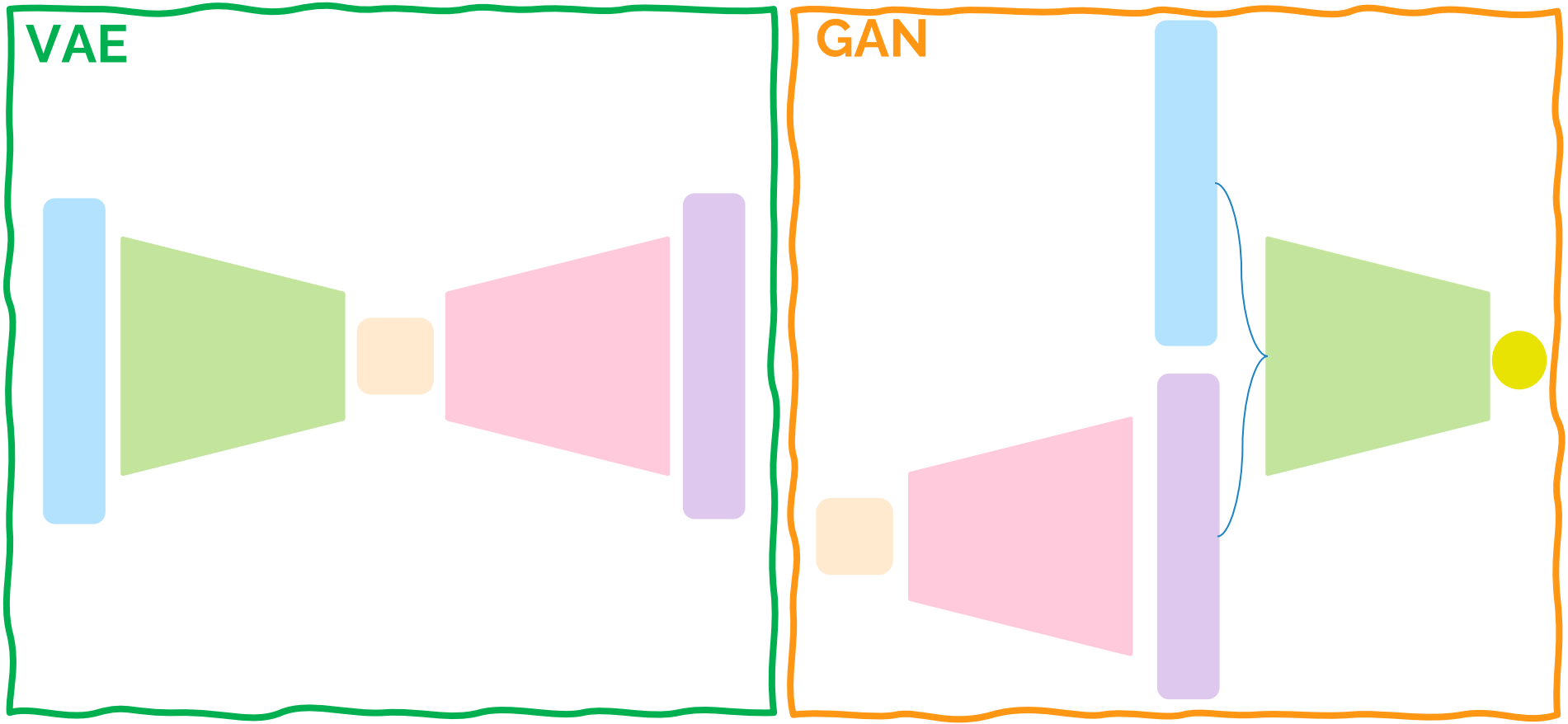
Generative Modeling: GAN and Diffusion Model

Mohammad Iqbal, S.Si., M.Si., Ph. D.

OUTLINES

- Recall: VAE and GAN
- *Generative Adversarial Networks*
- *Diffusion Model*
- *Diffusion Model Image Data*
- *Diffusion Model Text Data*

Latent Variable



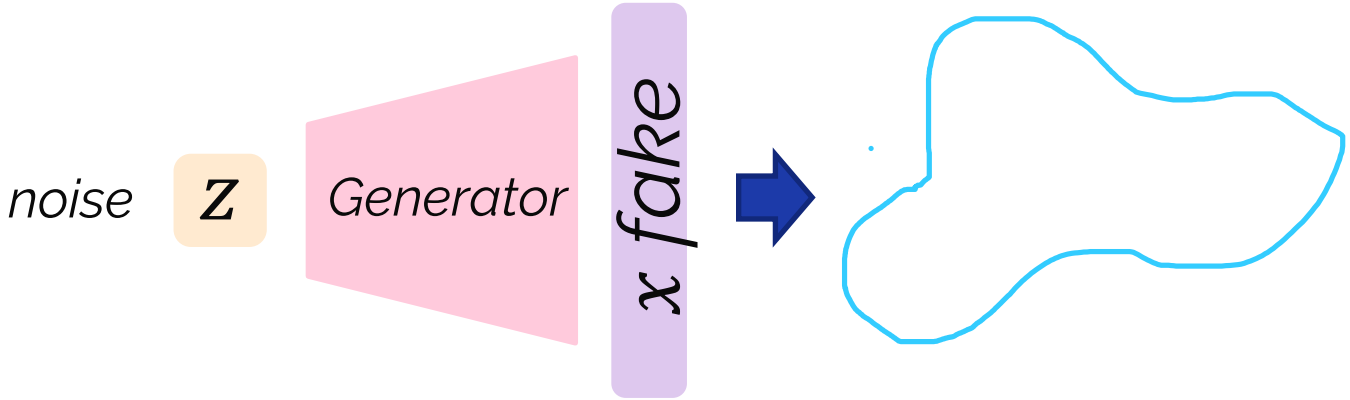
Generative Adversarial Networks

What if only want to sample?

Idea : don't explicitly model density and instead just sample a to generate new instances

Problem : want to sample from complex distribution – can't do this directly!

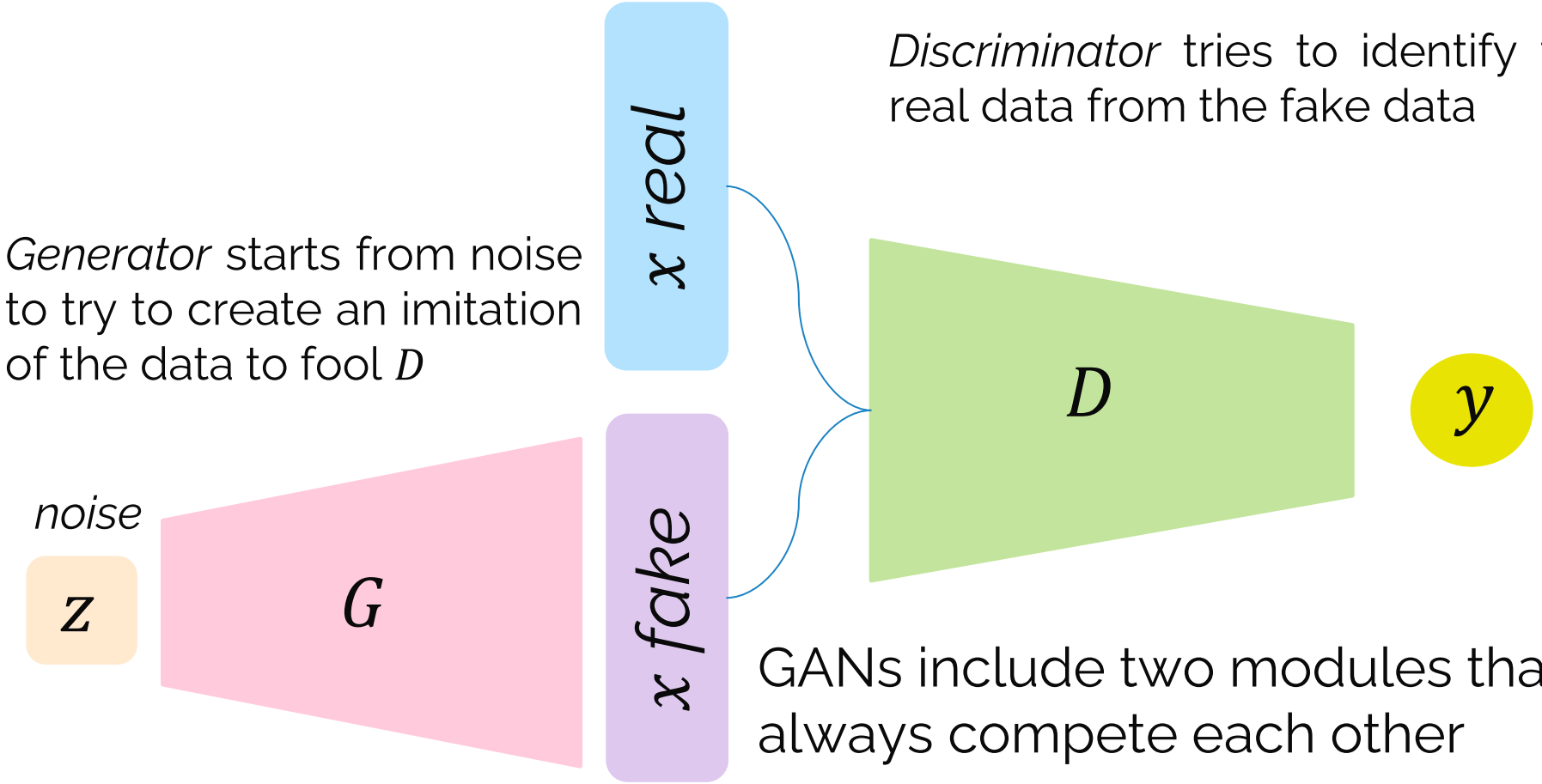
- Sampe from something simple (e.g., noise)
- Learn a transformation to the data distribution



Generative Adversarial Networks

Discriminator tries to identify the real data from the fake data

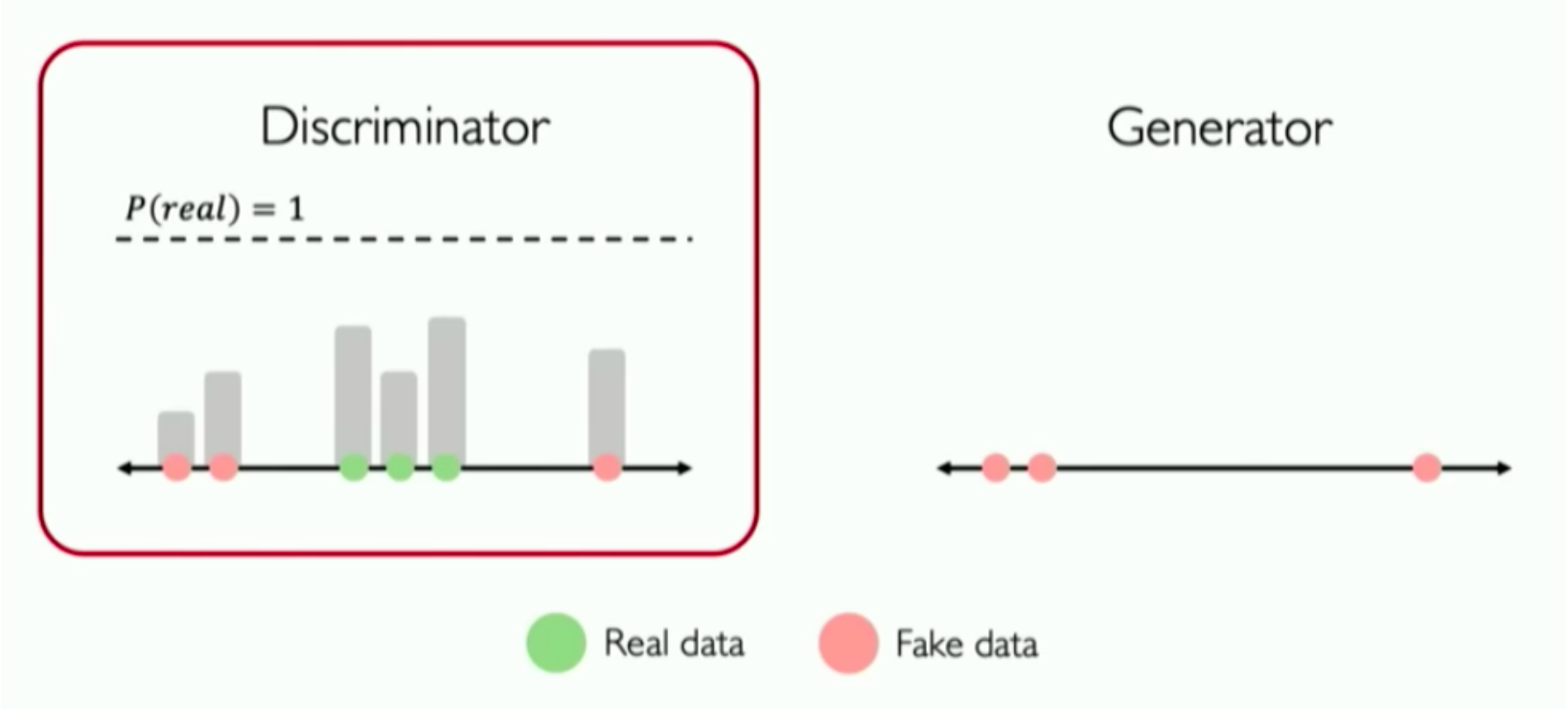
Generator starts from noise to try to create an imitation of the data to fool D



GANs include two modules that always compete each other

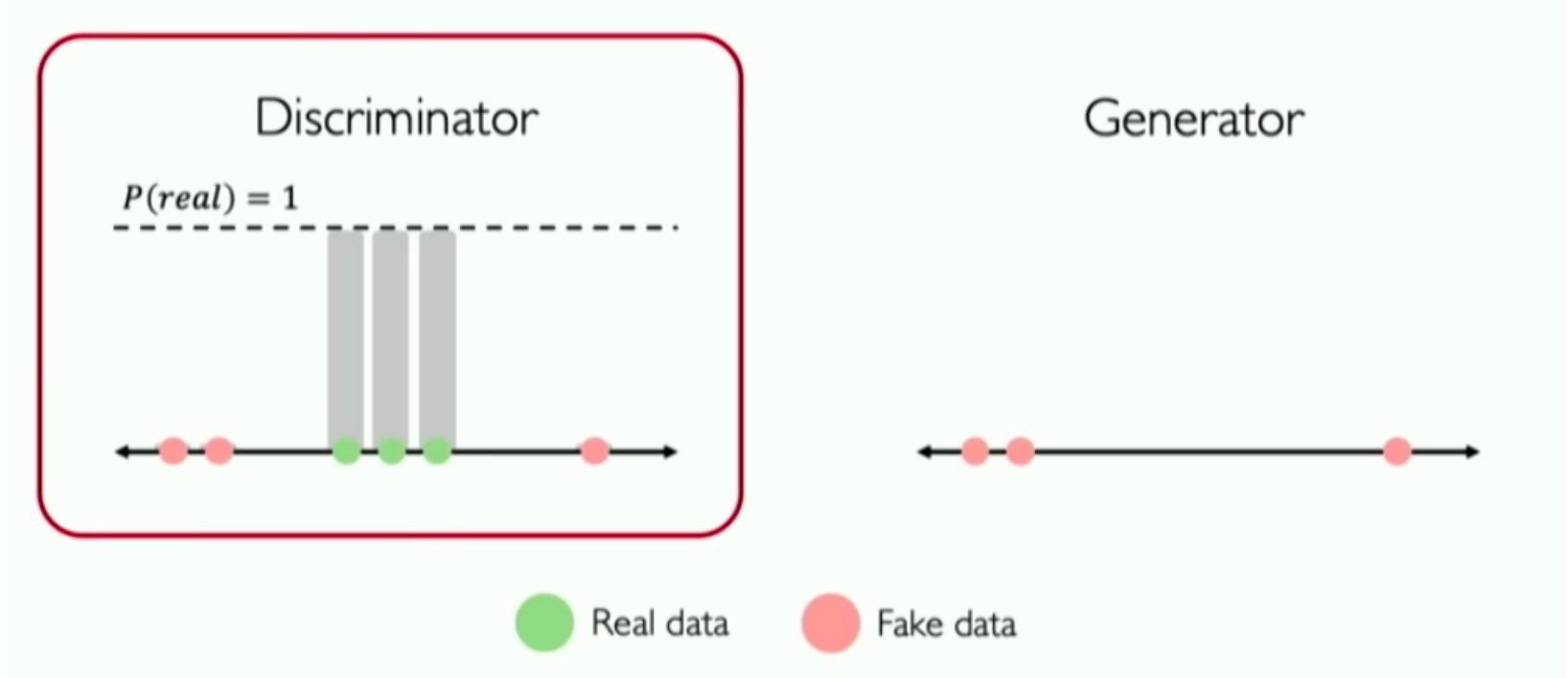
Discriminator tries to search the real!

http://introtodeeplearning.com



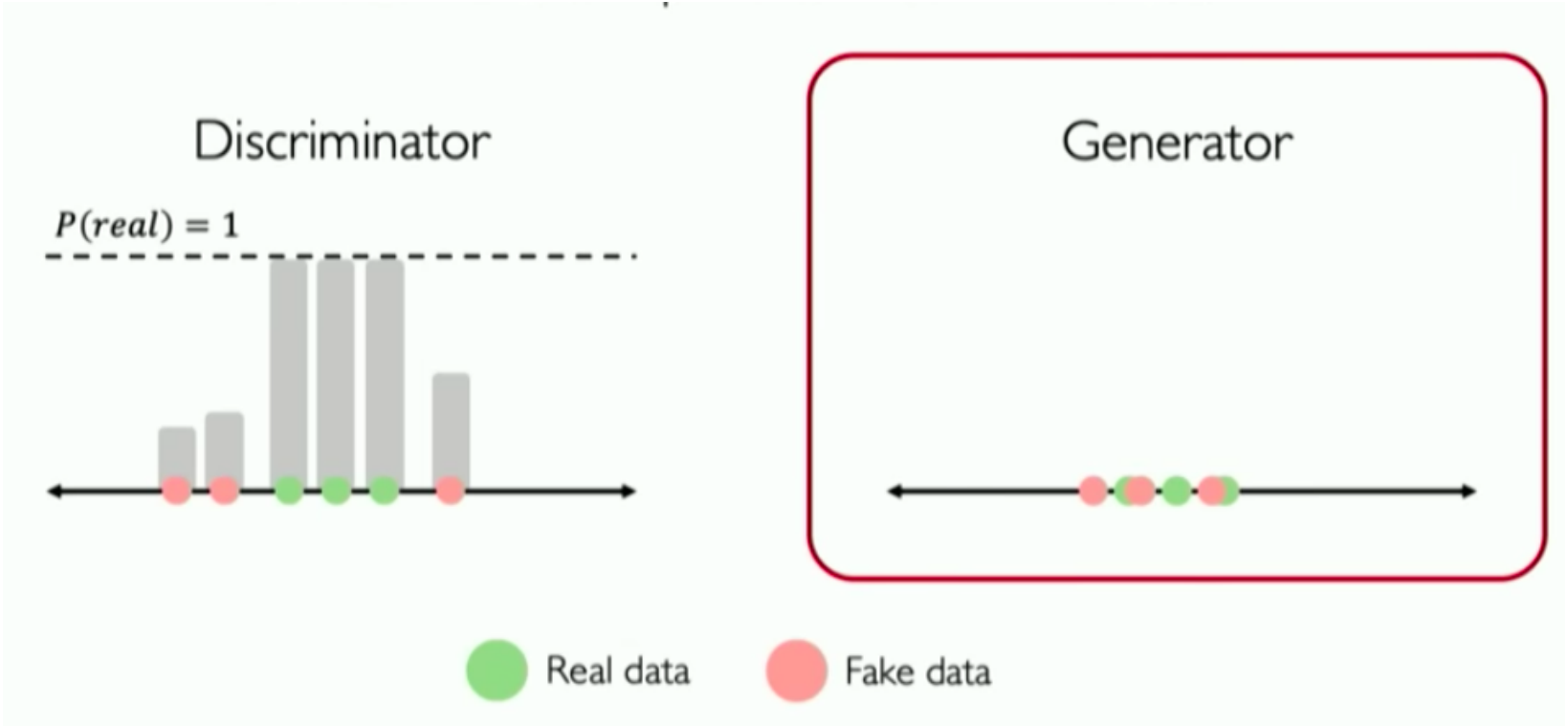
Discriminator tries to search the real!

http://introtodeeplearning.com

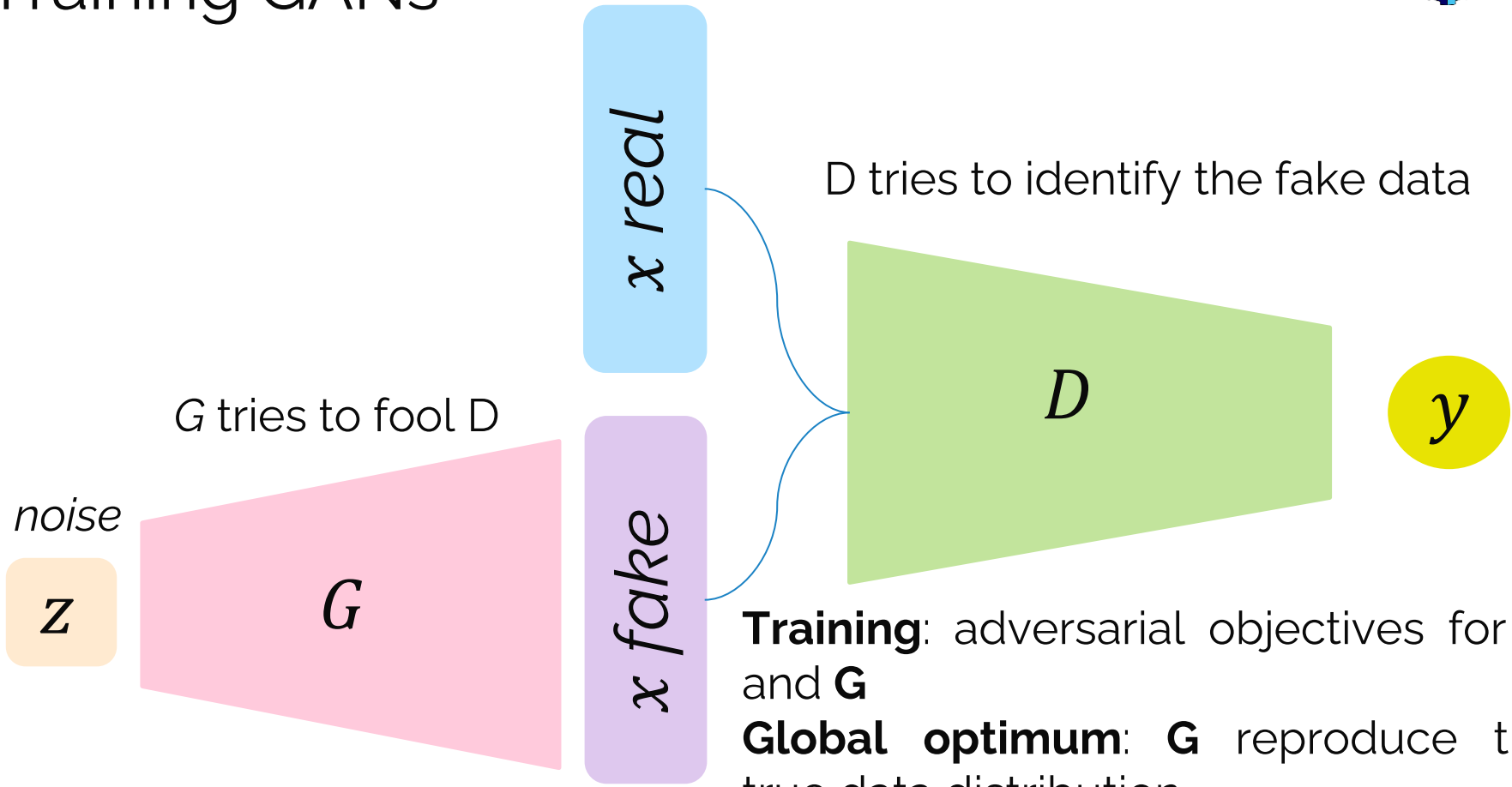


Generator tries to fake it!

http://introtodeeplearning.com



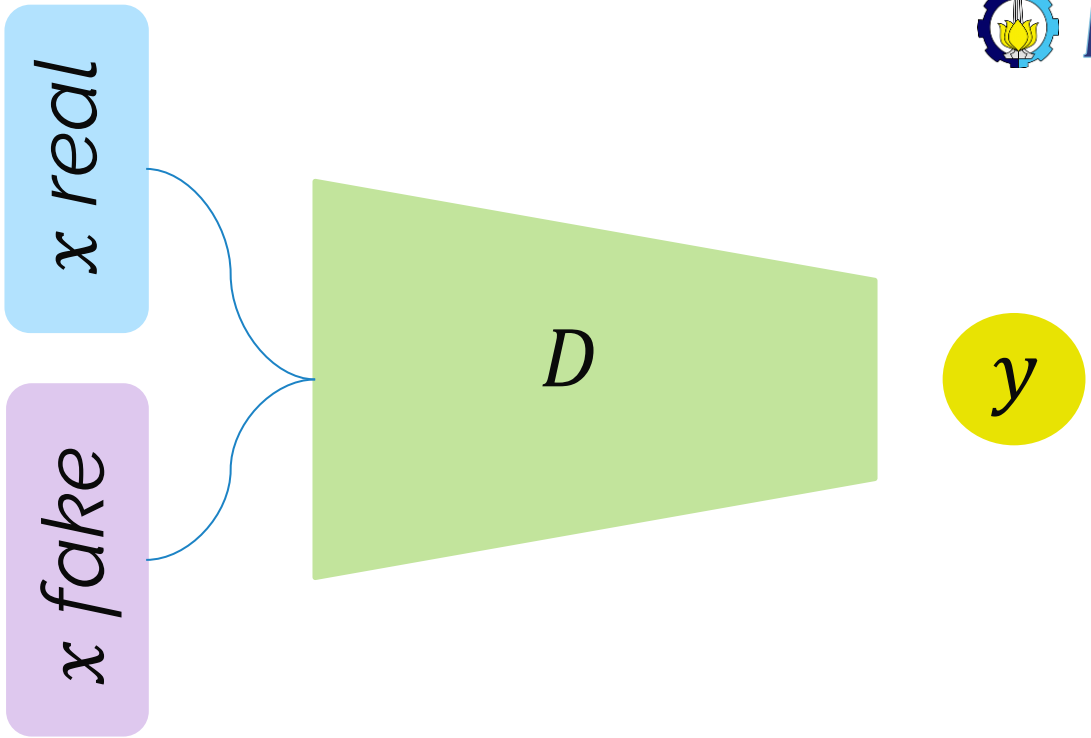
Training GANs



Training: adversarial objectives for **D** and **G**

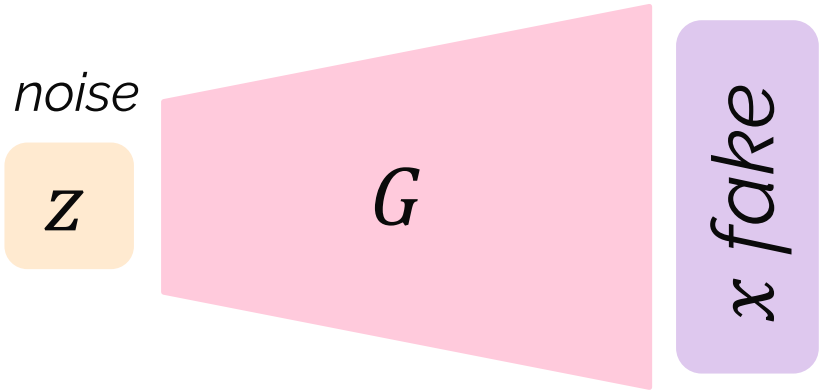
Global optimum: **G** reproduce the true data distribution

Training GANs



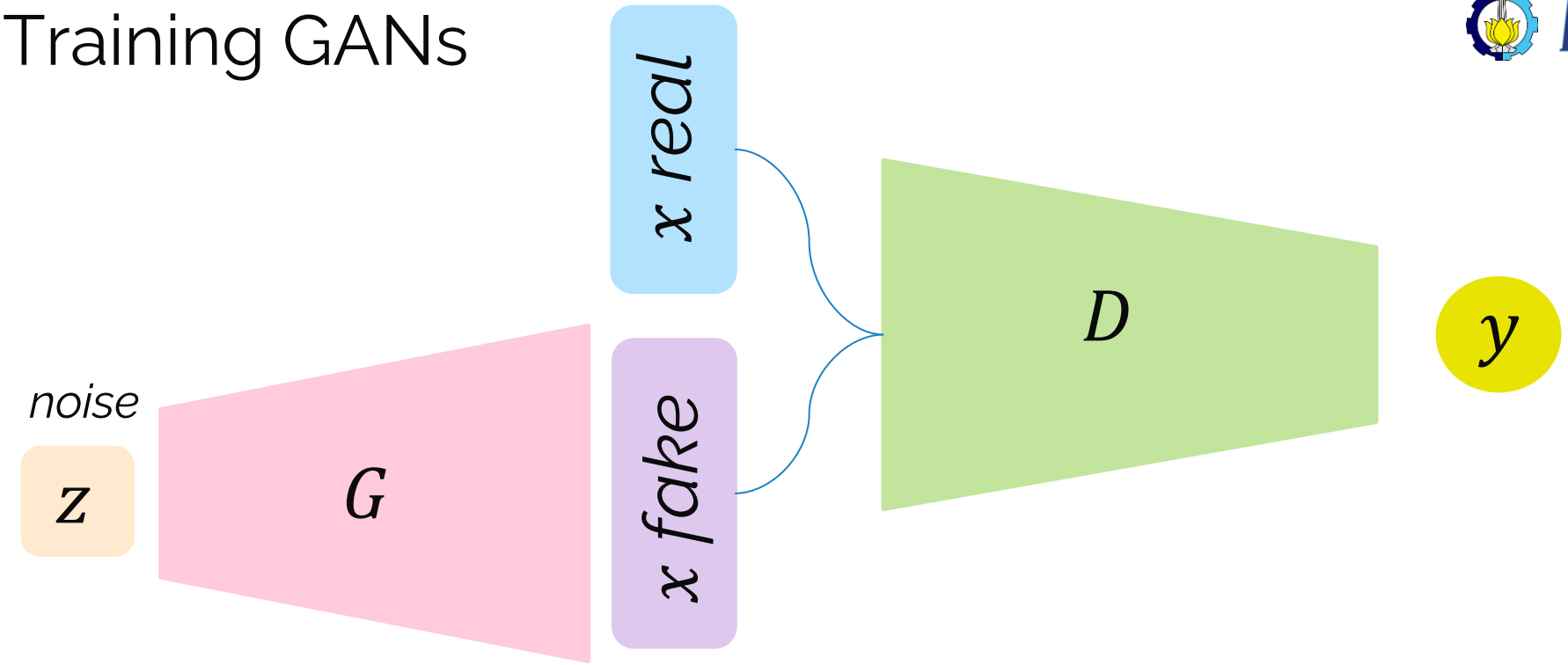
$$\underset{G}{\operatorname{argmin}} \mathbb{E}_{z,x}\left[\log D\left(G(z)\right) + \log\left(1 - D(x)\right)\right]$$

Training GANs



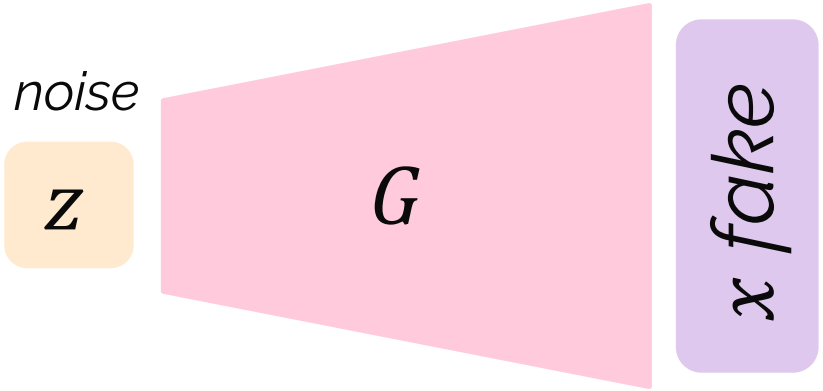
$$\underset{D}{\operatorname{argmax}}\, \mathbb{E}_{z,x}\left[\log D\left(G(z)\right) + \log\left(1 - D(x)\right)\right]$$

Training GANs



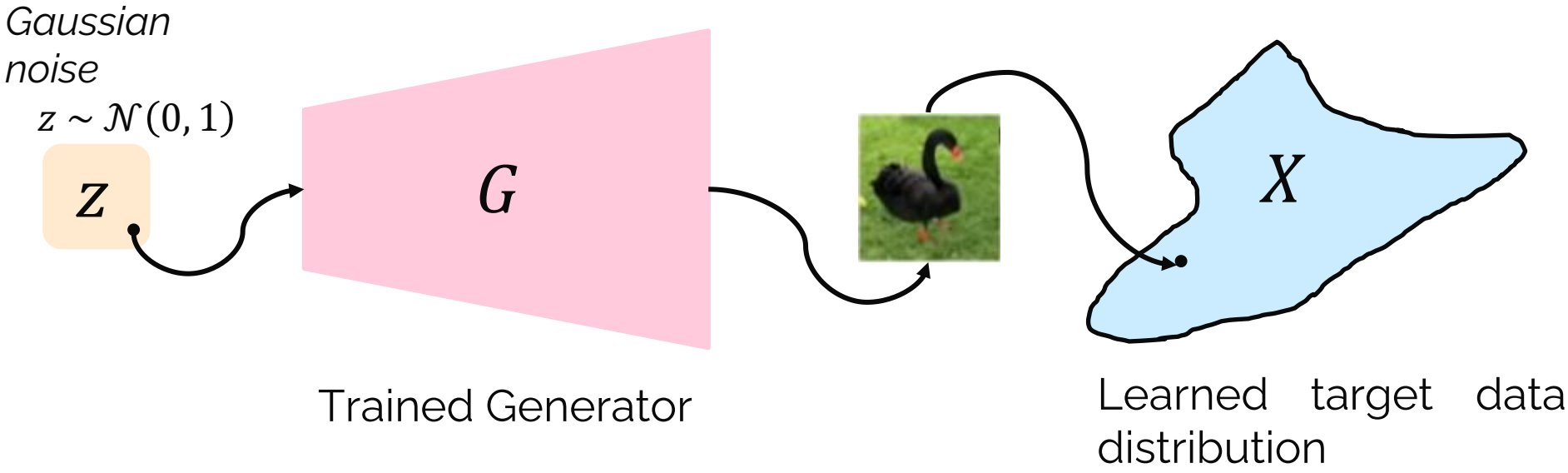
$$\arg\min_{G}\max_{D}\mathbb{E}_{z,x}\left[\log D\left(G(z)\right)+\log\left(1-D(x)\right)\right]$$

Generating new data

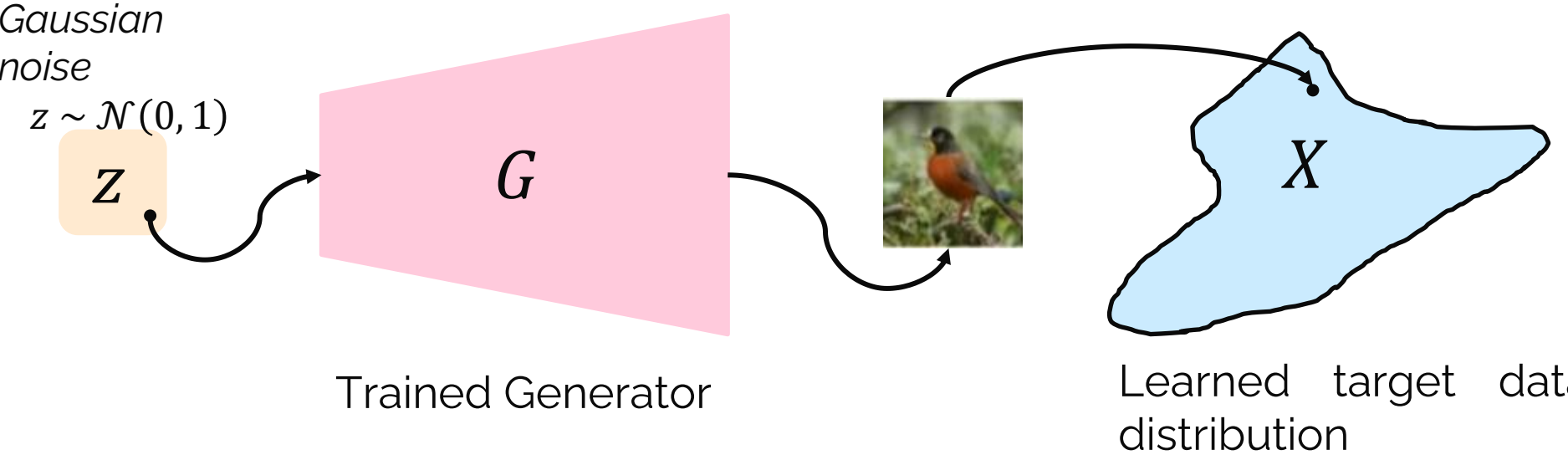


After training, use generator network to use **new data** that is never been seen before

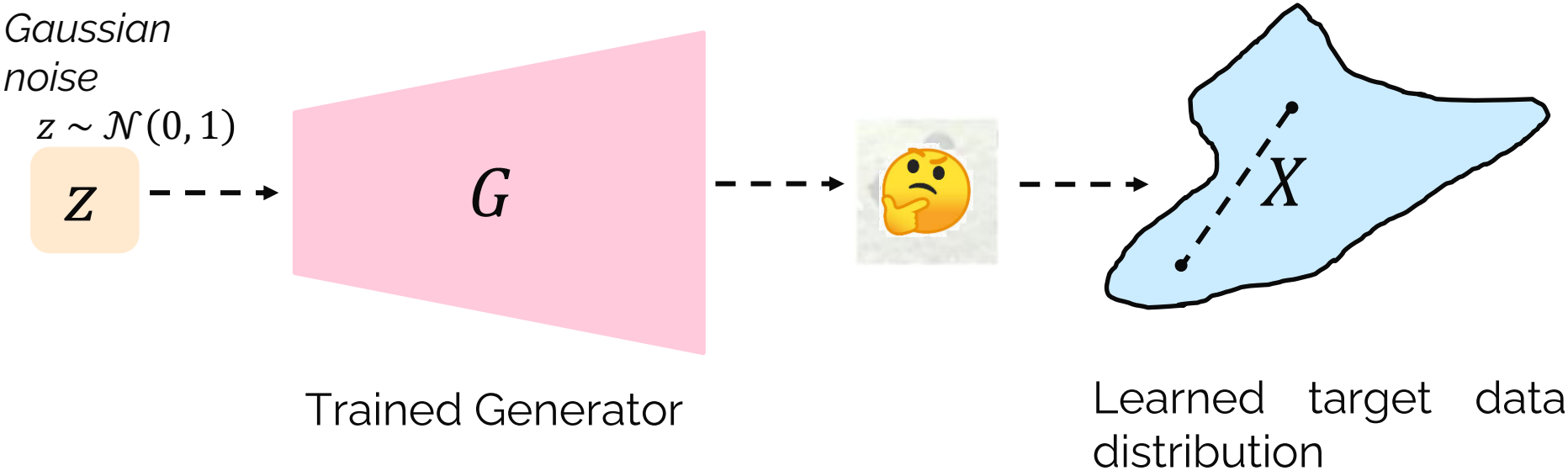
GANs are distribution transformer!



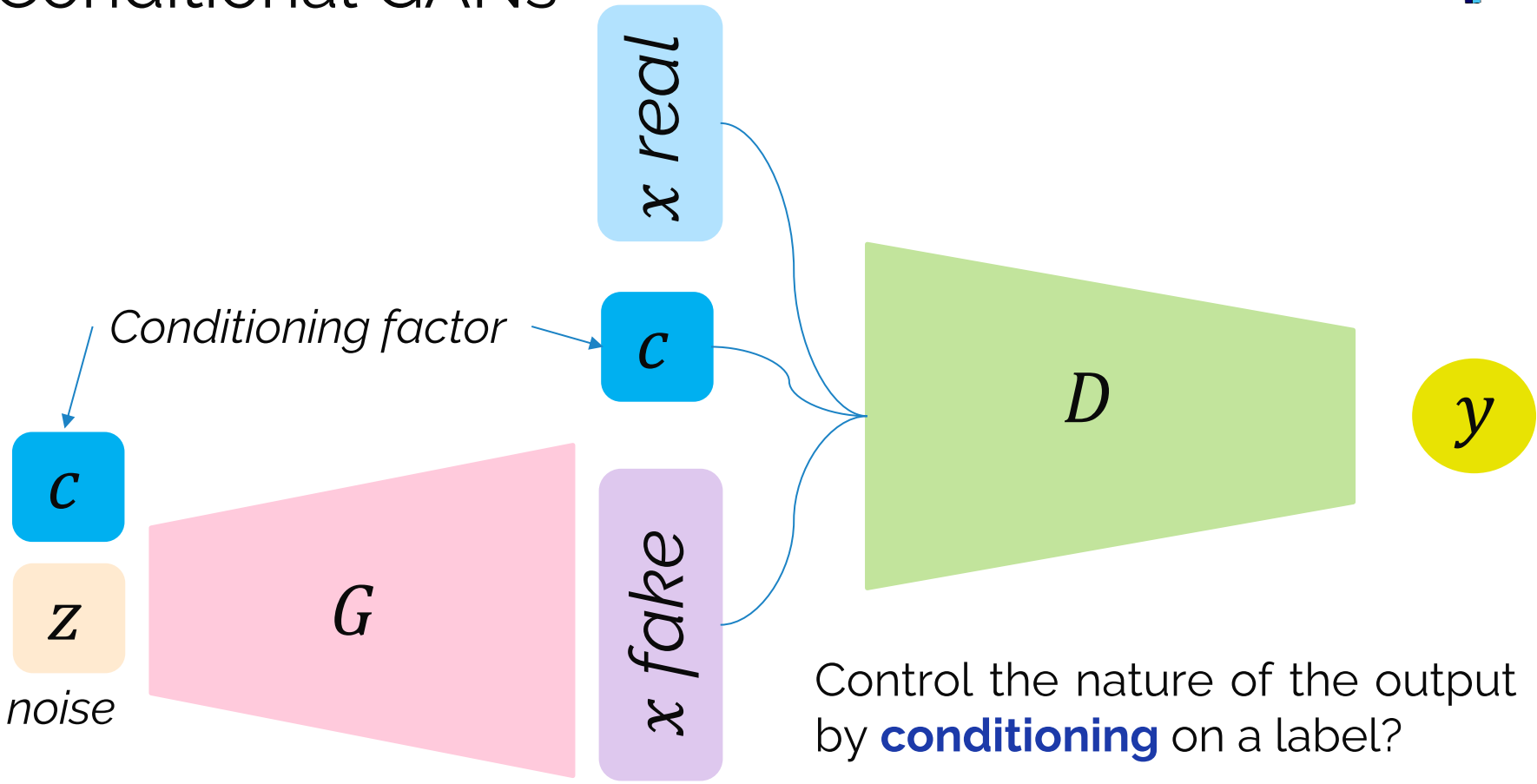
GANs are distribution transformer!



GANs merupakan distribusi transformer!



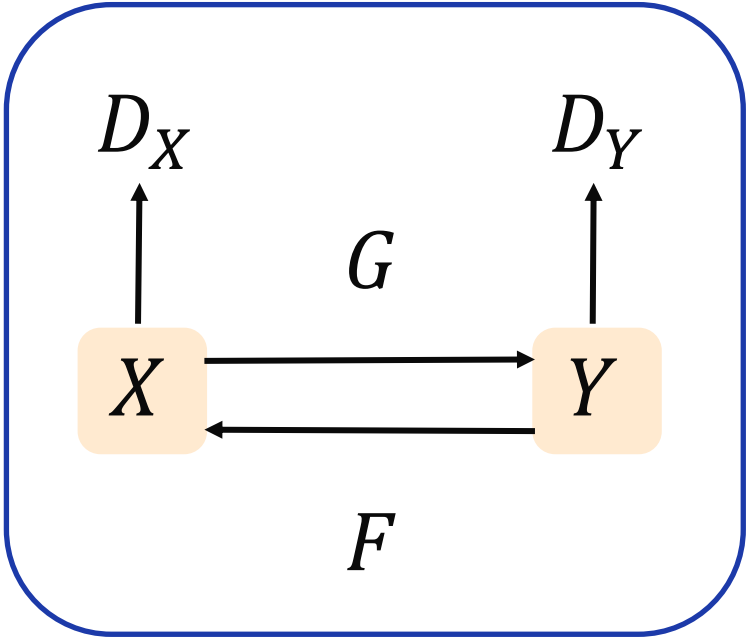
Conditional GANs



Control the nature of the output by **conditioning** on a label?

CycleGANs

CycleGAN learns transformations across domains with unpaired data.

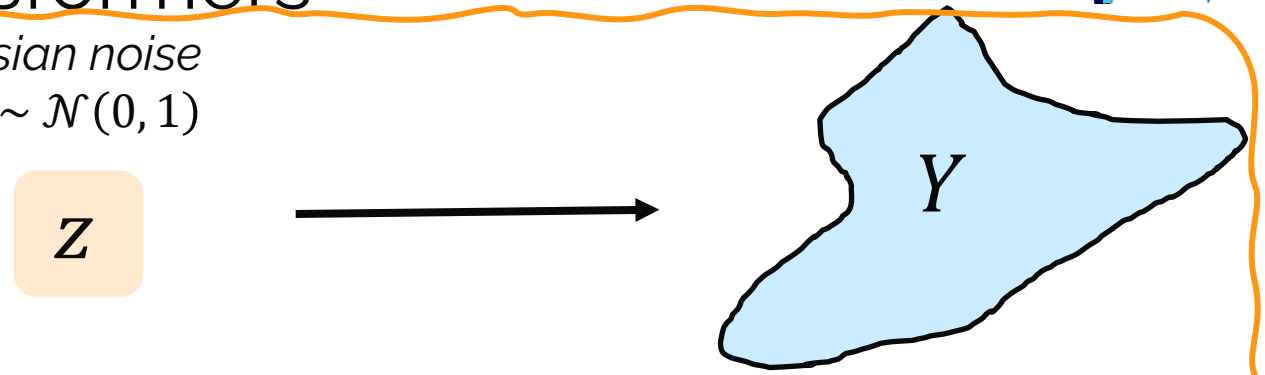


Distribution transformers

GANs :

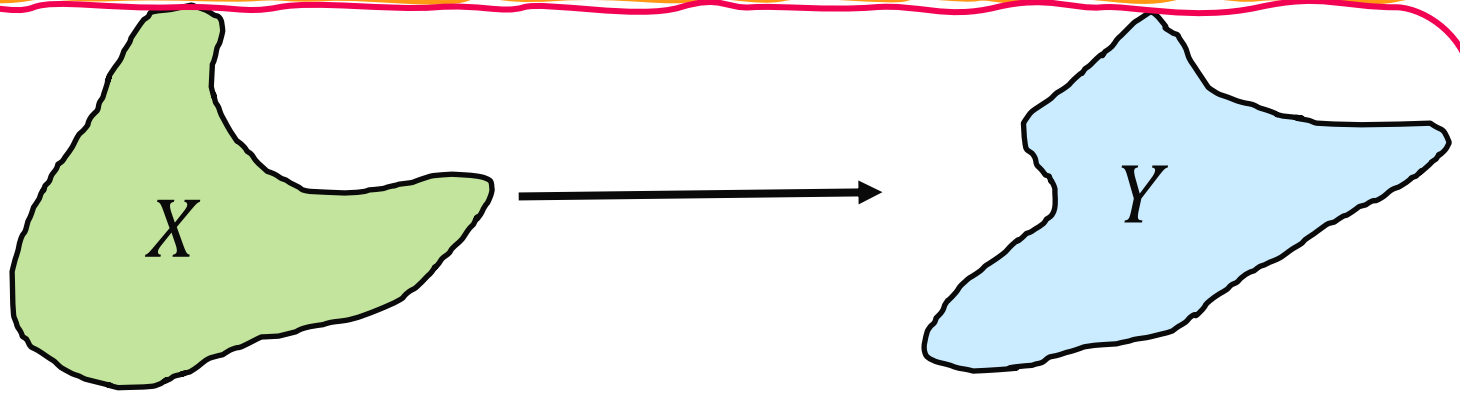
Gaussian noise

$z \sim \mathcal{N}(0, 1)$



Gaussian noise → manifold data target

CycleGANs :



manifold data X → manifold data Y

Applications (Cycle GAN)

Applications (DeepFAKE)

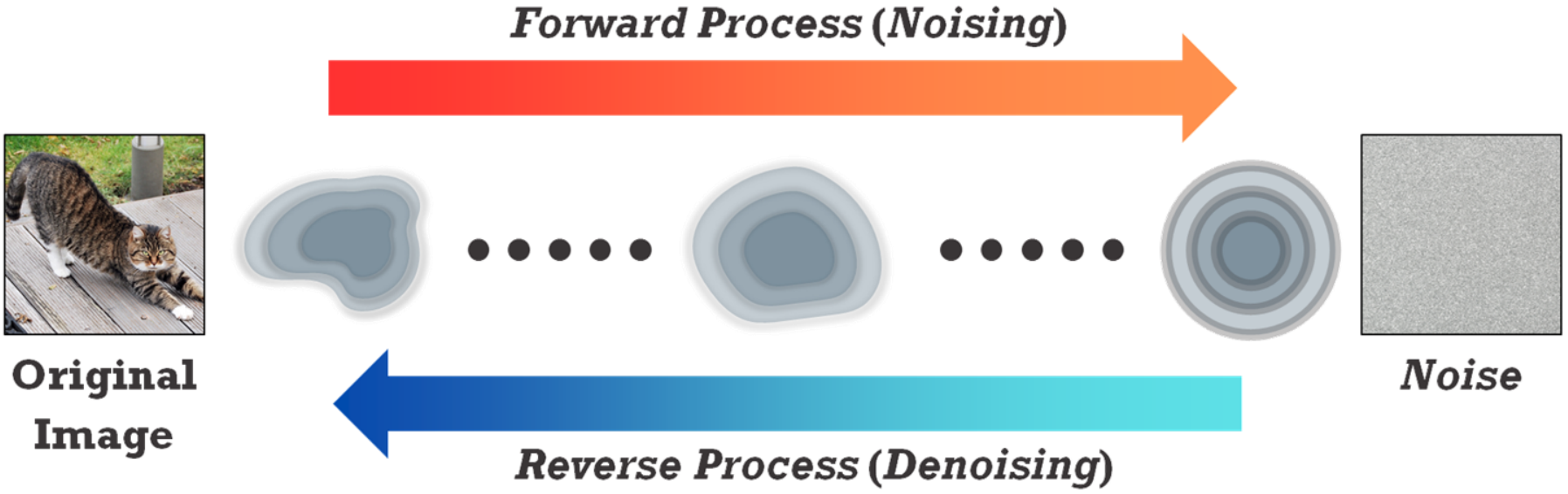
Diffusion Model

Diffusion Model

- Diffusion model is one of generative model
- Diffusion model has been successfully applied on Images
- The main goal is to generate image based on **noise**
- The task is to predict **the entire noise to be removed** in a given timestep

Diffusion Model: Overview

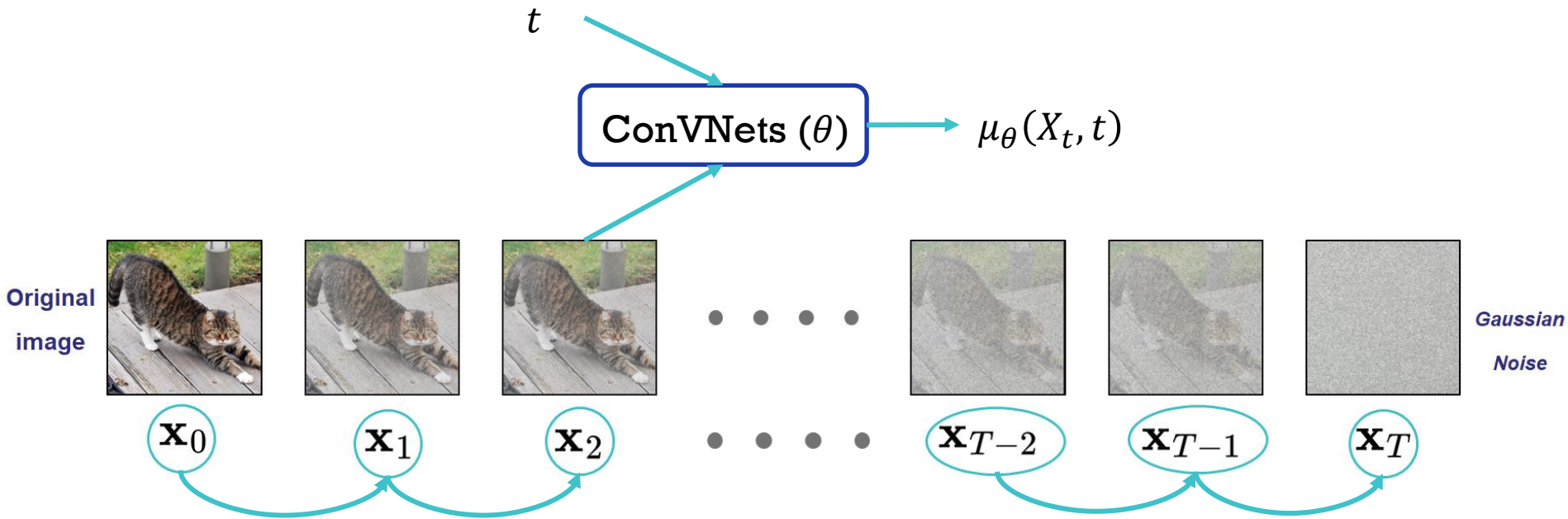
- Diffusion model is under **Gaussian Estimation**



Diffusion Model on Image Data

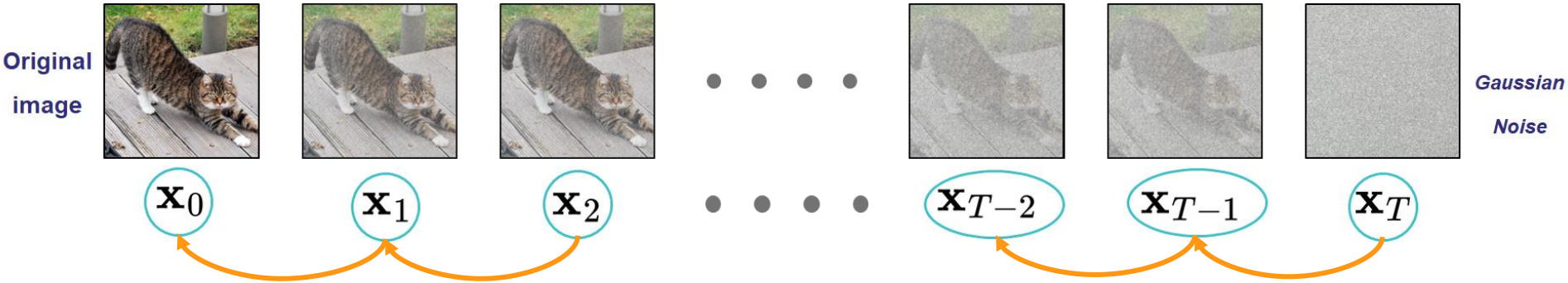
Diffusion Model: Forward Process

- Generative Process : $q_\theta(X_{T-1} \mid X_T)$



Diffusion Model: Denoising Process

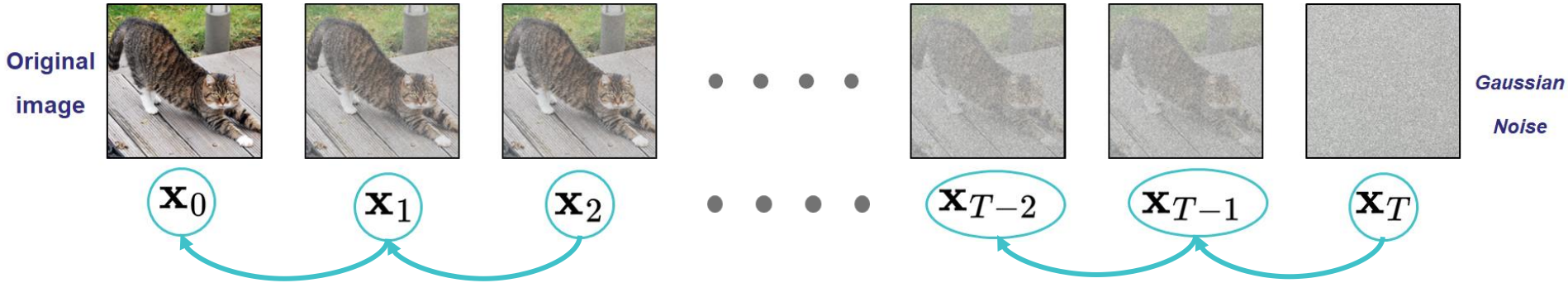
- **Training** : Construct latent variables pairs, then apply supervised training



- **Latent Construction**: $p_\phi(X_t \mid X_{t-1}) = \mathcal{N}\left(X_t; \sqrt{1-\beta_t} X_{t-1}, \beta_t I\right)$

Diffusion Model: Denoising Process

- **Training** : Construct latent variables pairs, then apply supervised training

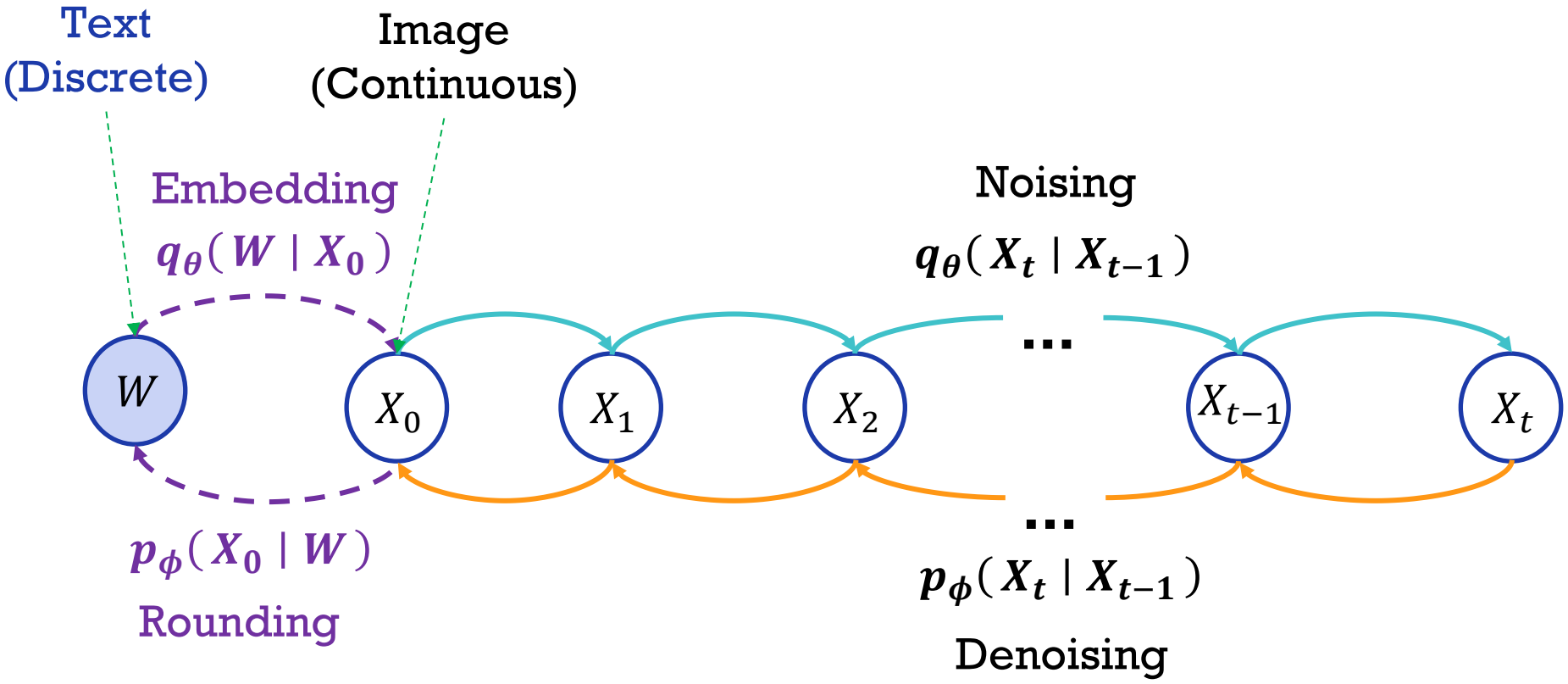


- **Supervised Training**: $\mathcal{L}_{\text{simple}}(X_0) = \sum_{t=1}^{T} \mathop{\mathbb{E}}_{p(X_t \mid X_0)} \|\mu_\theta(X_t, t) - \mu_{t-1}(X_t, X_0)\|^2$

where, $\mu_{t-1}(X_t, X_0) = \mathop{\mathbb{E}}_{p}[X_{t-1} \mid x_t = X_t, x_0 = X_0]$

Diffusion Model on Text Data

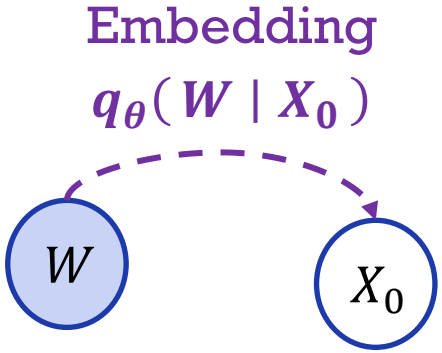
Diffusion on Text Data



Diffusion on Text Data: Embedding

Embed (w_i) maps each word to $\mathbb{R}^d$

Embed $(W) = [\textbf{Embbed}\ (w_1), \ldots, \textbf{Embbed}\ (w_n)] \in \mathbb{R}^d$



$$q_\theta(X_0 \mid W) = \mathcal{N}(\textbf{Embed}(W), \sigma_0 I)$$

where σ_0 is the hyperparameter (a small number)

and $X_0, .., X_T \in \mathbb{R}^{nd}$

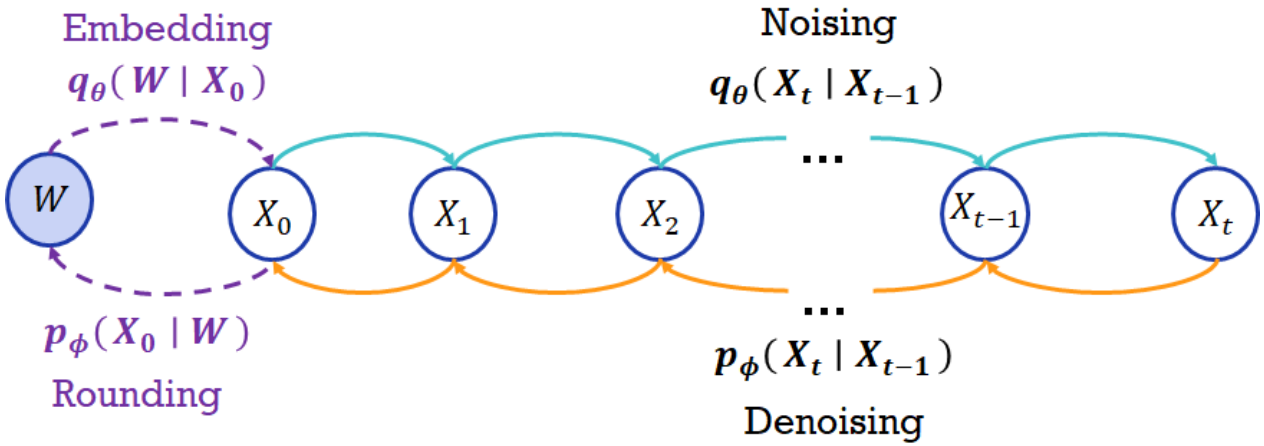
How to choose the embedding function Embed (w_i) ?

Embedding: End to End Training

Maximize probability of Embedding

$$\mathcal{L}_{e2e} = \mathop{\mathbb{E}}_{X_0 \sim q_\theta} \left[\mathcal{L}_{simple}(X_0) - \log p_\phi(W \mid X_0) \right]$$

Reconstruction loss



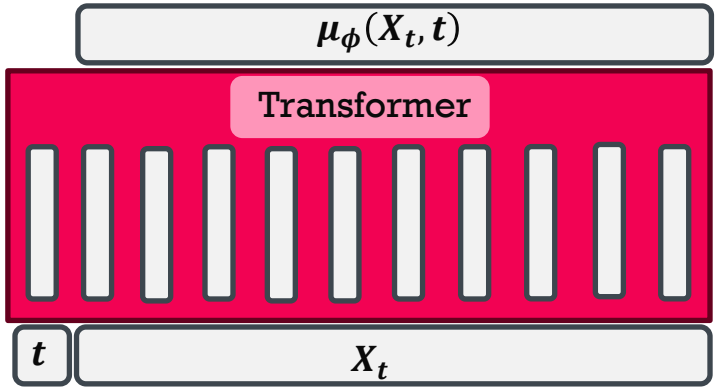
Embedding: End to End Training

Maximize probability of Embedding

$$\mathcal{L}_{e2e} = \mathop{\mathbb{E}}_{X_0 \sim q_\theta} \left[\mathcal{L}_{simple}(X_0) - \log p_\phi(W \mid X_0) \right]$$

Reconstruction loss

$$\mathcal{L}_{simple}(X_0) = \sum_{t=1}^{T} \mathop{\mathbb{E}}_{q_\theta(X_t|X_0)} \left\| \mu_\phi(X_t, t) - \mu_{t-1}(X_t, X_0) \right\|^2$$

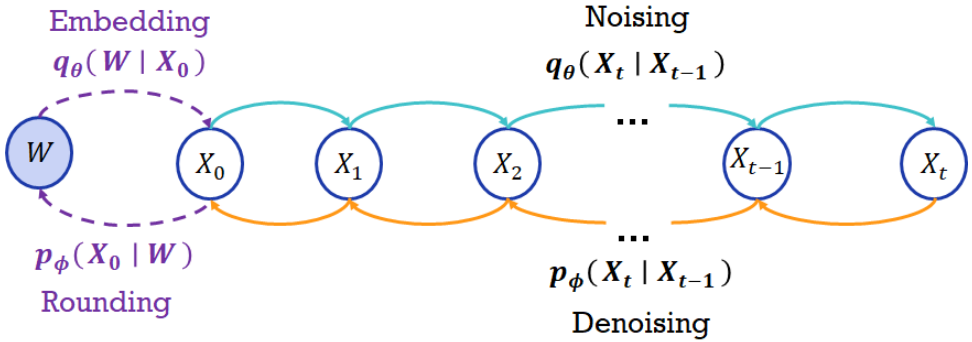


Embedding: End to End Training

Maximize probability of Embedding

$$\mathcal{L}_{e2e} = \mathop{\mathbb{E}}_{X_0 \sim p_\phi} \left[\mathcal{L}_{simple}(X_0) - \log p_\phi(W \mid X_0) \right]$$

Reconstruction loss



$$p_\phi(W \mid X_0) = \prod_{i=1}^{n} p_\phi(w_i \mid x_i)$$

Reducing Rounding Error

$$\widehat{w_i} = \underset{w_i}{\operatorname{argmax}}\, p_\phi(w_i \mid X_0[i])$$

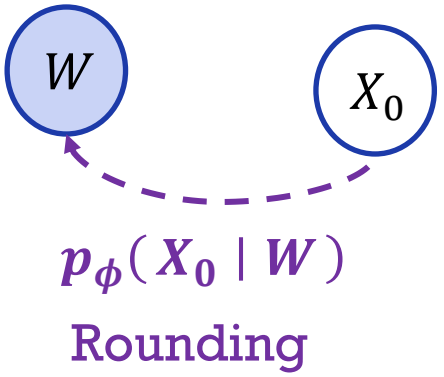
- **Ideally**, the model should predict X_0 that lies exactly on a word embedding
- **Reality**: There is still rounding error

My ice cream is [BLANK].

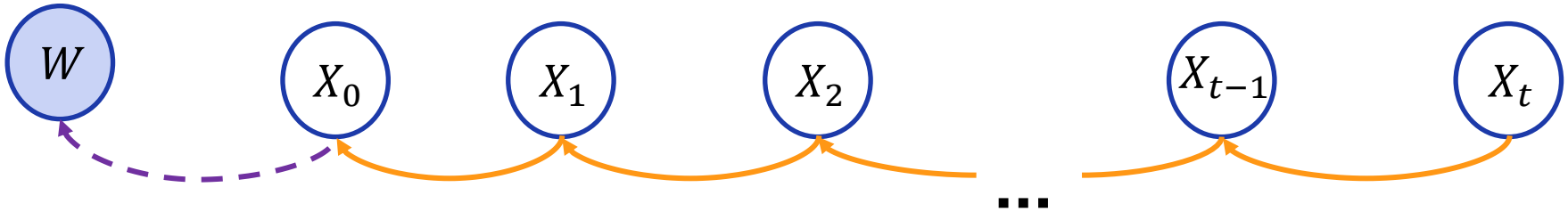
- Melting ✓
- Saving ✗

"Melting" and "Saving" are close in the embedding space.

But they are not substitutable in this context



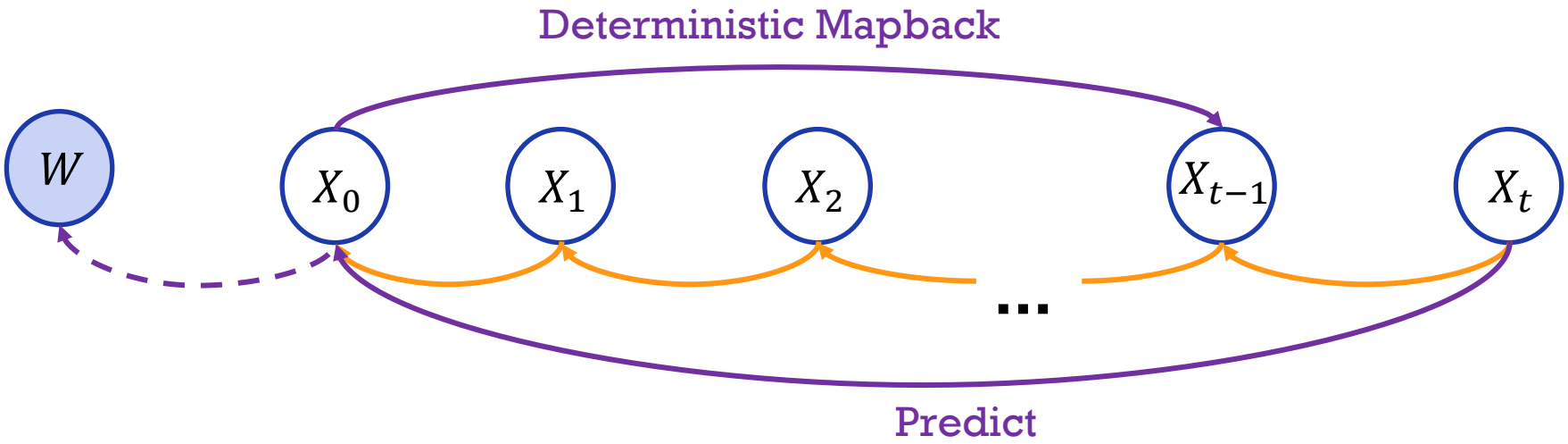
Reducing Rounding Error (training time)



$$\mathcal{L}_{simple}(X_0) = \sum_{t=1}^{T} \mathop{\mathbb{E}}_{q_\theta(X_t|X_0)} \left\| \mu_\phi(X_t, t) - \mu_{t-1}(X_t, X_0) \right\|^2$$

Intuition: Rounding happens all at the last step: $x_1 \rightarrow x_0$ which is hard and prone to error.

Reducing Rounding Error (training time)

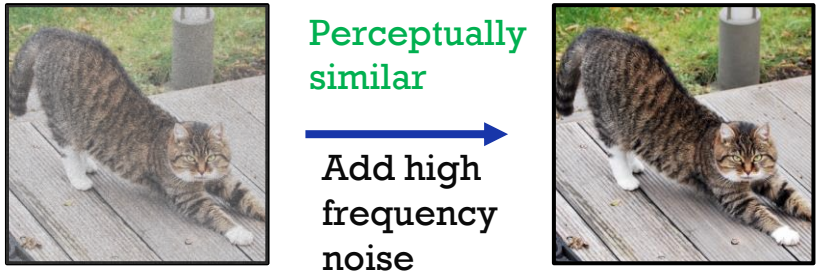


$$\mathcal{L}_{simple}(X_0) = \sum_{t=1}^{T} \mathop{\mathbb{E}}_{q_\theta(X_t|X_0)} \left\| f_\phi(X_t, t) - X_0 \right\|^2$$

Intuition: Training to predict X_0 at each diffusion step makes predicted X_0 mor precise and reduce rounding error.

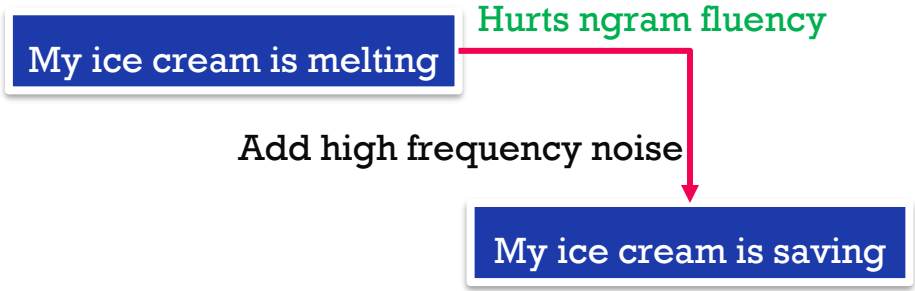
Additional: Image vs Text

- **Low frequency**: the large scale structure of the images.
- **Medium frequency**: the details.
- **High frequency**: Perceptually meaningless.



High noise levels: build up the low-frequency components
Low noise levels: refine high-frequency details.

- **Frequency**: the rate of change in the embedding of the neighboring words.
- High frequency components matters a lot for text because it relates to token prediction and it is easy to notice bad grams.



- The most natural way to capture high frequency components is to do next token prediction.
- Diffusion-LM still benefit from the low frequency (coarse-grained)

Diffusion vs Autoregressive

How **human** writes long text:
Core concepts → writing structure → wording and phrasing

Too different

How **autoregressive** LM produce Long text:
Left-to-right.

Closer

How **diffusion** LM produce long text:
Coarse-to-fine

Diffusion vs Autoregressive

	Summary
Training Efficiency	Autoregressive LM > Diffusion LM
Decoding Efficiency	Autoregressive LM > Diffusion LM O(sequence length) > O(number of different steps)
Flexible Generation Order	Autoregressive LM < Diffusion LM
Controllable Text Generation	Autoregressive LM < Diffusion LM

Discussion...